

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 5 :

Algorithmes de tri et de recherche (sur Python)

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Algorithmes de tri
 - a. Tri à bulle
 - b. Tri par insertion
 - c. Tri par sélection
 - d. Tri par fusion
2. Algorithmes de recherche
 - a. Recherche linéaire
 - b. Recherche par arbre binaire

Motivation

Motivation

Exemples d'applications des algorithmes de tri et de recherche:

- Fichiers Excel des notes des étudiants, on veut classer selon l'ordre
- Fichier Excel des notes des étudiants, on veut chercher la meilleure note
- Google pour chercher ce que vous demander
- Facebook pour classer les vidéos à vous recommander selon le plus intéressant pour vous
- **Ctrl + F** : chercher dans un document
- Etc.

Algorithmes de tri:

Tri à bulle

D'où vient le nom (pour bien comprendre) :

- Le tri à bulle tire son nom de la manière dont les éléments se déplacent à travers la liste lors du tri. L'algorithme parcourt répétitivement la liste, comparant les paires d'éléments adjacents et échangeant leurs positions si nécessaire, de sorte que les éléments plus petits (ou plus légers) "remontent" progressivement vers le début de la liste comme des bulles remontant à la surface de l'eau.

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié  
  
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

12	11	13	5	6
----	----	----	---	---

12 > 11 ? Oui
Echange les

Quand $i = 0$, $\text{range}(0, 5-0-1 = 4)$
 j donc va de 0 à 3

Maintenant, $j = 0$

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	12	13	5	6
----	----	----	---	---

Maintenant, $j = 1$

12 > 13 ? Non
Ne fait rien

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```


Comment ça marche, pas à pas

11	12	13	5	6
----	----	----	---	---

Maintenant, $j = 2$

13 > 5 ? Oui
Echange les

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	12	5	13	6
----	----	---	----	---

13 > 6 ? Oui
Echange les

Maintenant, $j = 3$

Et on finit ici, dernière itération,
 $n - i - 1 = 5 - 0 - 1 = 4$

Donc on finit à 3 (rappeler,
toujours $n-1$ dans les boucles)

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	12	5	6	13
----	----	---	---	----

Et voici le résultat de la première itération de i ($i=0$)

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié  
  
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	12	5	6	13
----	----	---	---	----



11 > 12 ? Non
Ne fait rien

Quand $i = 1$, $\text{range}(0, 5-1-1 = 3)$
 j donc va de 0 à 2

Maintenant, $j = 0$

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	12	5	6	13
----	----	---	---	----

Maintenant, $j = 1$

12 > 5 ? Oui
Echange les

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	5	12	6	13
----	---	----	---	----



12 > 6 ? Oui
Echange les

Maintenant, $j = 2$
Et on finit ici

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	5	6	12	13
----	---	---	----	----

Et voici le résultat de la
deuxième itération de i (i=1)

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié  
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

11	5	6	12	13
----	---	---	----	----

11 > 5 ? Oui
Echange les

Quand $i = 2$, $\text{range}(0, 5-2-1 = 2)$
 j donc va de 0 à 1

Maintenant, $j = 0$

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```


Comment ça marche, pas à pas

5	11	6	12	13
---	----	---	----	----

11 > 6 ? Oui
Echange les

Maintenant, $j = 1$
Et on finit ici

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr
```

```
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié
```

```
[5, 6, 11, 12, 13]
```

Comment ça marche, pas à pas

5	6	11	12	13
---	---	----	----	----

Et voici le résultat de la troisième itération de i ($i=2$)

On doit continuer de même façon pour $i=3$ et $i=4$

Pour ce cas, la liste est déjà triée, donc la comparaison de `arr[j] > arr[j+1]` sera fausse, et on n'échangera pas les valeurs

```
def tri_bulles(arr):
    n = len(arr)

    for i in range(n):
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier
        for j in range(0, n - i - 1):
            # Parcourez le tableau de 0 à n-i-1
            # Échangez si l'élément trouvé est supérieur à l'élément suivant
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

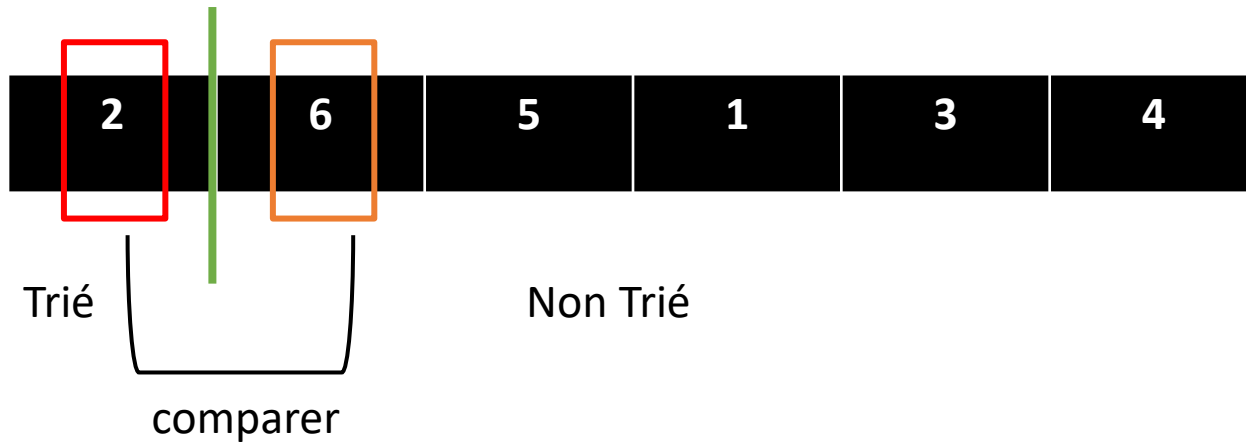
a = [12, 11, 13, 5, 6]
a_trié = tri_bulles(a)
a_trié
```

[5, 6, 11, 12, 13]

Algorithmes de tri:

Tri par insertion

Comment ça marche, pas à pas



L'idée du tri par insertion est que nous considérons la liste autant que deux parties : partie triée et partie non triée.

Avant la première itération, la partie triée ne contient que le premier élément, Puis, l'objectif est d'étendre la partie triée en insérant les éléments de la partie non triée d'une manière correcte.

```
for i in range(1, len(arr)): ## donc i va de 1 à len(arr)
    i = 1
    j = i # j = 1
```

```
def tri_insertion(arr):
    for i in range(1, len(arr)):
        j = i

        while arr[j-1] > arr[j] and j > 0:
            arr[j-1], arr[j] = arr[j], arr[j-1]
            j -= 1

    return arr

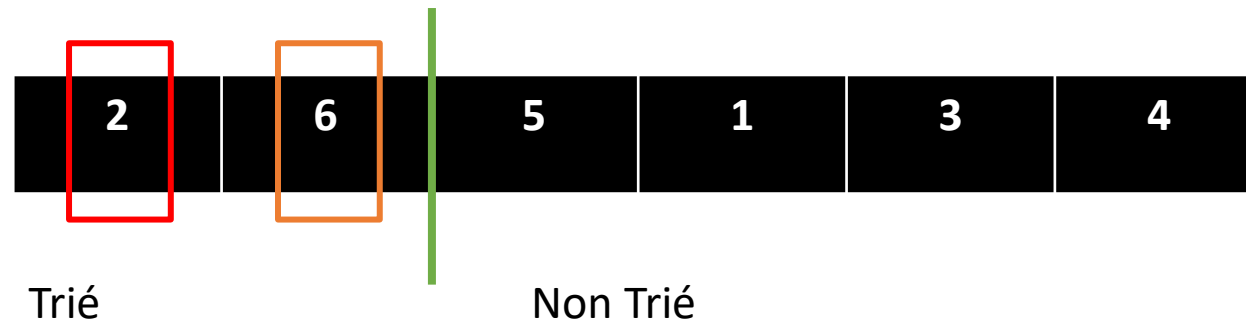
a = [22, 31, 13, 15, 1]
a_trié = tri_insertion(a)
print(a_trié)

[1, 13, 15, 22, 31]
```

Comment ça marche, pas à pas



Résultat :



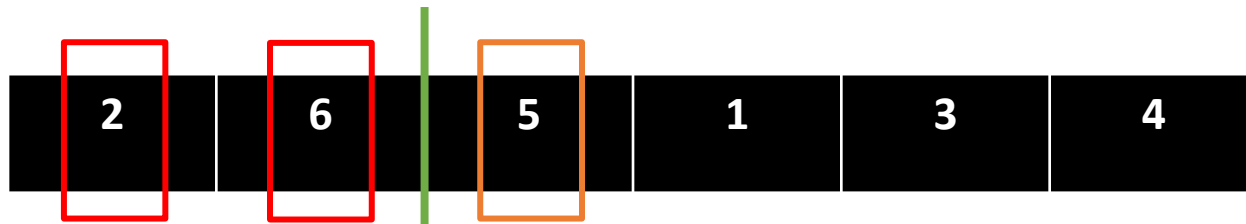
```
for i in range(1, len(arr)): ## donc i va de 1 à len(arr)
    i = 1
    j = i # j = 1
```

```
def tri_insertion(arr):
    for i in range(1, len(arr)):
        j = i
        while arr[j-1] > arr[j] and j > 0:
            arr[j-1], arr[j] = arr[j], arr[j-1]
            j -= 1
    return arr
```

```
a = [22, 31, 13, 15, 1]
a_trié = tri_insertion(a)
print(a_trié)
```

```
[1, 13, 15, 22, 31]
```

Comment ça marche, pas à pas



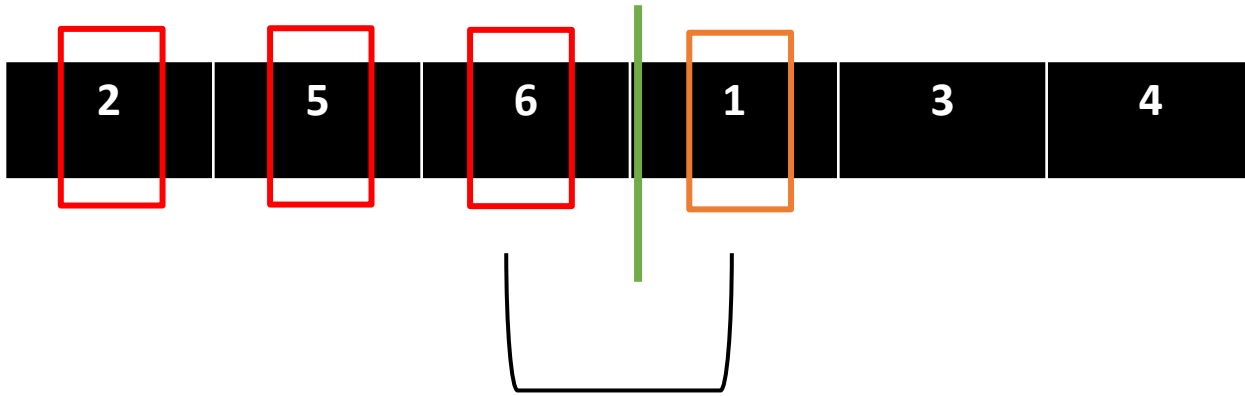
```
for i in range(1, len(arr)): ## donc i va de 1 à len(arr)
    i = 2
    j = i # j = 2
```

```
def tri_insertion(arr):
    for i in range(1, len(arr)):
        j = i
        while arr[j-1] > arr[j] and j > 0:
            arr[j-1], arr[j] = arr[j], arr[j-1]
            j -= 1
    return arr

a = [22, 31, 13, 15, 1]
a_trié = tri_insertion(a)
print(a_trié)
```

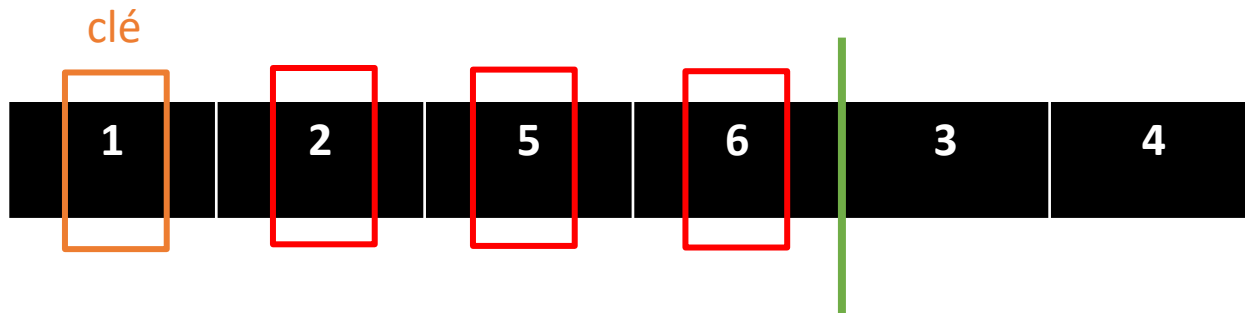
[1, 13, 15, 22, 31]

Comment ça marche, pas à pas



On compare donc le premier élément de la partie non triée avec tous les éléments de la partie triée comme précédemment

Résultat :



On continue comme ça jusqu'à ce que la liste soit complètement triée

```
for i in range(1, len(arr)): ## donc i va de 1 à len(arr)
    i = 3
    j = i # j = 3
```

```
def tri_insertion(arr):
    for i in range(1, len(arr)):
        j = i
        while arr[j-1] > arr[j] and j > 0:
            arr[j-1], arr[j] = arr[j], arr[j-1]
            j -= 1
    return arr
```

```
a = [22, 31, 13, 15, 1]
a_trié = tri_insertion(a)
print(a_trié)
```

```
[1, 13, 15, 22, 31]
```

Algorithmes de tri:

Tri par sélection

L'algorithme de tri par sélection, cherche la valeur minimum, et essaye de la pousser vers la gauche de la liste.

```
def tri_selection(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        # Trouver l'élément minimum dans la partie non triée du tableau  
        indice_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[indice_min]:  
                indice_min = j  
  
        # Échanger l'élément minimum trouvé avec le premier élément  
        arr[i], arr[indice_min] = arr[indice_min], arr[i]  
  
    return arr  
  
a = [10, 7, 17, 25, 2]  
a_trié = tri_selection(a)  
print(a_trié)  
  
[2, 7, 10, 17, 25]
```

Ce bloc du code qui trouve le min de la partie non triée de la liste

Comment ça marche, pas à pas

Liste à considérer



$i = 0$
indice_min = i

min

Résultat itération $i=0$:



L'idée est donc de :

- trouver le minimum dans la liste
- placer le en premier
- refaire la même procédure, pour la liste commençant cette fois-ci après le minimum que nous venons de trouver

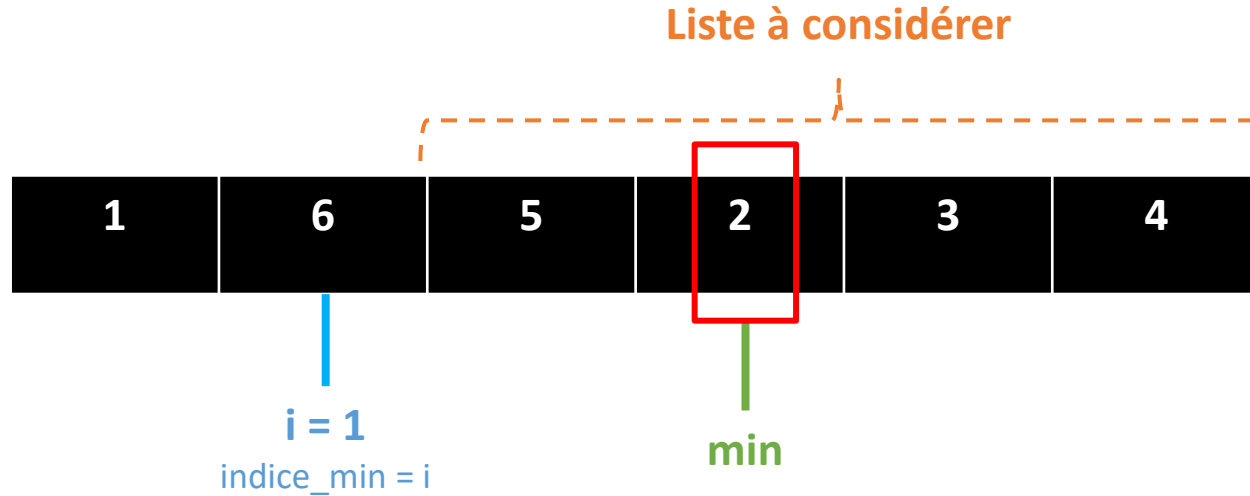
```
def tri_selection(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        # Trouver l'élément minimum dans la partie non triée du tableau  
        indice_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[indice_min]:  
                indice_min = j  
  
        # Échanger l'élément minimum trouvé avec le premier élément  
        arr[i], arr[indice_min] = arr[indice_min], arr[i]  
  
    return arr
```

Ce bloc du code qui
trouve le min de la partie
non triée de la liste

```
a = [10, 7, 17, 25, 2]  
a_trié = tri_selection(a)  
print(a_trié)
```

```
[2, 7, 10, 17, 25]
```

Comment ça marche, pas à pas



Résultat itération $i=1$:

1	2	5	6	3	4
---	---	---	---	---	---

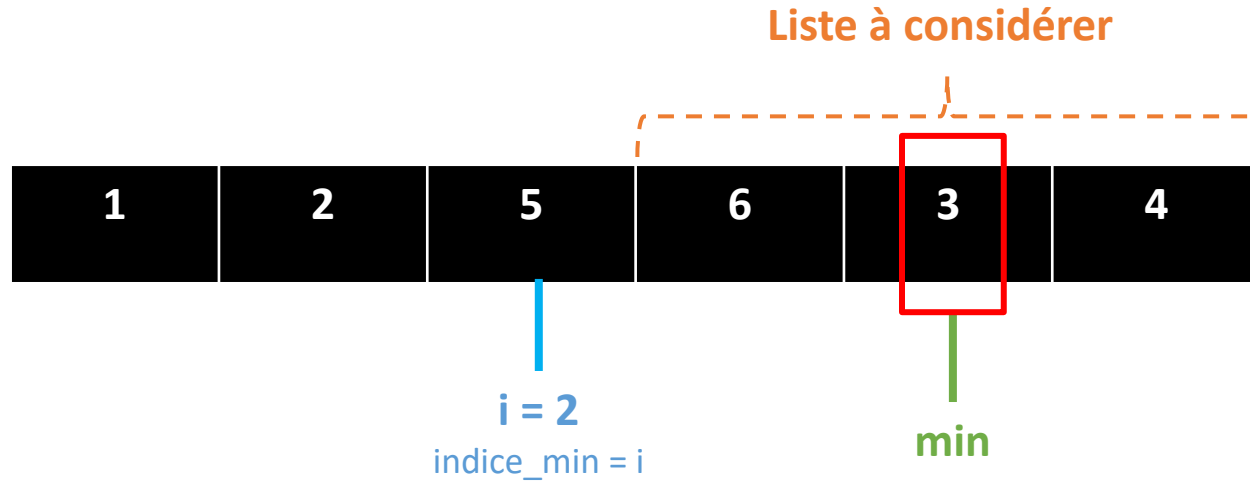
L'idée est donc de :

- trouver le minimum dans la liste
- placer le en premier
- refaire la même procédure, pour la liste commençant cette fois-ci après le minimum que nous venons de trouver

```
def tri_selection(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        # Trouver l'élément minimum dans la partie non triée du tableau  
        indice_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[indice_min]:  
                indice_min = j  
  
        # Échanger l'élément minimum trouvé avec le premier élément  
        arr[i], arr[indice_min] = arr[indice_min], arr[i]  
  
    return arr  
  
a = [10, 7, 17, 25, 2]  
a_trié = tri_selection(a)  
print(a_trié)  
  
[2, 7, 10, 17, 25]
```

Ce bloc du code qui trouve le min de la partie non triée de la liste

Comment ça marche, pas à pas



Résultat itération $i=2$:



L'idée est donc de :

- trouver le minimum dans la liste
- placer le en premier
- refaire la même procédure, pour la liste commençant cette fois-ci après le minimum que nous venons de trouver

```
def tri_selection(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        # Trouver l'élément minimum dans la partie non triée du tableau  
        indice_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[indice_min]:  
                indice_min = j  
  
        # Échanger l'élément minimum trouvé avec le premier élément  
        arr[i], arr[indice_min] = arr[indice_min], arr[i]  
  
    return arr  
  
a = [10, 7, 17, 25, 2]  
a_trié = tri_selection(a)  
print(a_trié)  
  
[2, 7, 10, 17, 25]
```

Ce bloc du code qui trouve le min de la partie non triée de la liste

Comment ça marche, pas à pas

On continue comme ça, jusqu'à ce que la liste soit complètement triée

```
def tri_selection(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        # Trouver l'élément minimum dans la partie non triée du tableau  
        indice_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[indice_min]:  
                indice_min = j  
  
        # Échanger l'élément minimum trouvé avec le premier élément  
        arr[i], arr[indice_min] = arr[indice_min], arr[i]  
  
    return arr  
  
a = [10, 7, 17, 25, 2]  
a_trié = tri_selection(a)  
print(a_trié)  
  
[2, 7, 10, 17, 25]
```

1	2	3	4	5	6
---	---	---	---	---	---

Algorithmes de tri:

Tri par fusion

```
# -----  
# L'ordre d'exécution dans les fonctions récursives,  
# à comprendre avant de d'analyser l'algorithme de tri par fusion :
```

```
def fct(n):  
    if n<=1:  
        return 1  
    else:  
        fct(n-1)  
        print('bonjour :', n)
```

fct(4)

bonjour : 2
bonjour : 3
bonjour : 4

Qui peut m'écrire
l'analyse de l'exécution
de ce programme ?

Exécution :

```
fct(4)  # 1er appel  
|  
v  
fct(3) 3 c'est n-1  ## et print('bonjour :', 4), 4 c'est n, sera mise en attente de revenir de l'appel fct(3)  
|  
v  
fct(2) 2 c'est n-1  ## et print('bonjour :', 3), 3 c'est n? sera mise en attente de revenir de l'appel fct(2)  
|  
v  
fct(1) 1 c'est n-1  ## et print('bonjour :', 2), 2 c'est n, sera mise en attente de revenir de l'appel fct(1)  
|  
v  
## ceci retourne 1,
```

et donc fct(1) vient de terminer, on print donc bonjour 2

puis, on remonte vers fct(2) qui vient de terminer aussi,
et on print bonjour 3

puis, on remonte vers fct(3) qui vient de terminer aussi,
et on print bonjour 4

Diviser et trier en utilisant la récursivité.

Cet algorithme est plus rapide que les trois autres algorithmes de tri précédents.

A vous d'analyser pourquoi !!
(on le verra dans la séance prochaine)

```
def tri_fusion(arr):
    if len(arr) > 1:
        milieu = len(arr) // 2

        moitie_gauche = arr[:milieu + len(arr) % 2] # Prend la première moitié de la liste
                                                    # inclut le milieu si la longueur de la liste est impaire

        moitie_droite = arr[milieu + len(arr) % 2:] # Prend la deuxième moitié de la liste

        tri_fusion(moitie_gauche) # Appel récursif pour trier la moitié gauche
        tri_fusion(moitie_droite) # Appel récursif pour trier la moitié droite

        i = j = k = 0
        ## 1ère boucle while : affecte à arr[0] = Le petit élément
                                #   arr[1] = Le petit élément dans ce qui reste
                                #   arr[2] = Le petit élément dans ce qui reste,
                                #   etc.
        while i < len(moitie_gauche) and j < len(moitie_droite):
            if moitie_gauche[i] < moitie_droite[j]:
                arr[k] = moitie_gauche[i]
                i += 1
            else:
                arr[k] = moitie_droite[j]
                j += 1
            k += 1

        # Les deuxième et troisième boucles while sont utilisées pour gérer les cas
        # où une des moitiés (gauche ou droite) aurait encore des éléments restants
        # après la fusion de la première boucle. Elles sont nécessaires car les moitiés
        # peuvent ne pas avoir la même longueur. Ces boucles ajoutent simplement les
        # éléments restants à la fin de la liste arr

        while i < len(moitie_gauche):
            arr[k] = moitie_gauche[i]
            i += 1
            k += 1

        while j < len(moitie_droite):
            arr[k] = moitie_droite[j]
            j += 1
            k += 1

    return arr
```


Tri par fusion

Démonstration : pour la séance prochaine